

SUPABASE: BACKEND PRONTO PARA APLICAÇÕES WEB

BANCO DE DADOS, AUTENTICAÇÃO, API E
PRIMEIRO CRUD

Tiago Henrique Angioletti





OBJETIVOS DA AULA

Ao final da aula, você deve conseguir:

- explicar o que é o Supabase;
- criar um projeto e entender sua estrutura;
- criar uma tabela no banco PostgreSQL;
- consultar dados pela interface do Supabase;
- usar a API gerada automaticamente;
- entender o papel das policies de segurança.



O PROBLEMA QUE O SUPABASE RESOLVE

Em uma aplicação web, normalmente precisamos de:

- banco de dados;

- API para ler e gravar dados;

- autenticação de usuários;

- regras de segurança;

- armazenamento de arquivos;

- painel para administrar os dados.

O Supabase entrega boa parte disso pronto.



O QUE É SUPABASE?

Supabase é uma plataforma de backend como serviço.

Na prática, ele oferece:

POSTGRESQL como banco principal;

API REST automática;

REALTIME para dados em tempo real;

AUTHENTICATION para login/cadastro;

STORAGE para arquivos;

EDGE FUNCTIONS para lógica server-side.



SUPABASE NÃO É SÓ “UM BANCO ONLINE”

O banco é PostgreSQL, mas o Supabase adiciona uma camada de ferramentas:

- painel visual para gerenciar tabelas;

- documentação automática da API;

- chaves de acesso;

- biblioteca JavaScript;

- segurança por linha com RLS;

- integração rápida com front-end.



ESTRUTURA DE UM PROJETO

Um projeto Supabase normalmente tem:

DATABASE: tabelas, colunas e relacionamentos;

TABLE EDITOR: edição visual dos registros;

SQL EDITOR: comandos SQL;

AUTHENTICATION: usuários e sessões;

API DOCS: exemplos de consulta;

PROJECT SETTINGS: URL e chaves do projeto.



EXEMPLO DE CENÁRIO DA AULA

Vamos imaginar uma aplicação simples de tarefas.

Cada tarefa terá:

Campo	Tipo	Exemplo
id	uuid	gerado automaticamente
titulo	text	Estudar Supabase
concluida	boolean	false
created_at	timestamp	data de criação



CRIANDO A TABELA PELO SQL EDITOR

```
create table tarefas (  
  id uuid primary key default gen_random_uuid(),  
  titulo text not null,  
  concluida boolean not null default false,  
  created_at timestamp with time zone default now()  
);
```

Esse comando cria a estrutura inicial para salvar tarefas.



INSERINDO DADOS DE TESTE

```
insert into tarefas (titulo, concluida)
values
  ('Criar projeto no Supabase', true),
  ('Criar tabela tarefas', true),
  ('Conectar com JavaScript', false);
```

Depois disso, os dados aparecem no **TABLE EDITOR**.



CONSULTANDO DADOS COM SQL

```
select *  
from tarefas  
order by created_at desc;
```

Também podemos filtrar:

```
select *  
from tarefas  
where concluida = false;
```



API AUTOMÁTICA

Quando criamos a tabela, o Supabase gera uma API.

Exemplo conceitual de rota:

```
GET /rest/v1/tarefas
```

Mas no front-end normalmente usamos a biblioteca oficial.



INSTALANDO A BIBLIOTECA JAVASCRIPT

Em projetos com npm:

```
npm install @supabase/supabase-js
```

Em uma página simples, também é possível usar CDN:

```
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>
```



CONECTANDO NO PROJETO

```
const SUPABASE_URL = 'https://seu-projeto.supabase.co';  
const SUPABASE_ANON_KEY = 'sua-chave-publica-anon';  
  
const client = supabase.createClient(  
  SUPABASE_URL,  
  SUPABASE_ANON_KEY  
);
```

A chave `anon` é pública, mas as regras de segurança precisam estar corretas.



LISTANDO TAREFAS COM JAVASCRIPT

```
async function carregarTarefas() {  
  const { data, error } = await client  
    .from('tarefas')  
    .select('*')  
    .order('created_at', { ascending: false });  
  
  if (error) {  
    console.error(error);  
    return;  
  }  
  
  console.log(data);  
}
```



CRIANDO UMA NOVA TAREFA

```
async function criarTarefa(titulo) {  
  const { data, error } = await client  
    .from('tarefas')  
    .insert({ titulo })  
    .select();  
  
  if (error) {  
    alert('Erro ao salvar tarefa');  
    return;  
  }  
  
  console.log('Tarefa criada:', data[0]);  
}
```



ATENÇÃO: RLS E SEGURANÇA

RLS significa **ROW LEVEL SECURITY**.

Ela controla quais linhas podem ser lidas, criadas, alteradas ou removidas.

Sem policies corretas, podem acontecer dois problemas:

- a API bloqueia tudo;
- ou libera dados demais.

Segurança no Supabase não é opcional.



EXEMPLO DE POLICY PARA LEITURA PÚBLICA

```
alter table tarefas enable row level security;  
  
create policy "Permitir leitura pública"  
on tarefas  
for select  
using (true);
```

Essa policy permite leitura pública.

Para sistemas reais, as regras devem considerar usuário logado, dono do registro e permissões.



PRÁTICA EM SALA

Crie um projeto Supabase e faça:

- criar a tabela `tarefas` ;

- inserir pelo menos 3 registros;

- consultar os registros no SQL Editor;

- abrir a documentação automática da API;

- localizar a URL do projeto e a chave `anon` ;

- discutir quais operações deveriam ser públicas ou privadas.



EXERCÍCIO

Monte uma página HTML simples que:

- tenha um campo para digitar uma tarefa;
- tenha um botão **ADICIONAR**;
- liste as tarefas vindas do Supabase;
- permita marcar uma tarefa como concluída.

Desafio: separar tarefas pendentes e concluídas visualmente.



REVISÃO DA AULA

Nesta aula vimos:

- o que é Supabase;
- como o PostgreSQL entra no projeto;
- criação de tabela com SQL;
- API automática;
- conexão com JavaScript;
- importância da RLS e das policies;
- caminho para criar um CRUD simples.